

《AI技术应用验证报告》

刘瑞雪 2022141090116

一、验证概述

1.1 验证目标

目标维度	具体内容	验证指标
效率提升	验证AI工具在测试用例编写上的时间节省效果	用例编写耗时对比
质量保障	验证AI生成测试用例的有效性与覆盖率	用例通过率、分支覆盖率
可落地性	验证AI工具与现有开发流程的集成可行性	工具集成耗时、使用流畅度

1.2 验证范围

本次验证聚焦于HomePick项目的**秒杀抢购核心功能**，包括：

- 秒杀库存预检（Redis）
- 一人一单限制（Redisson分布式锁）
- Lua脚本原子扣减
- RocketMQ异步下单

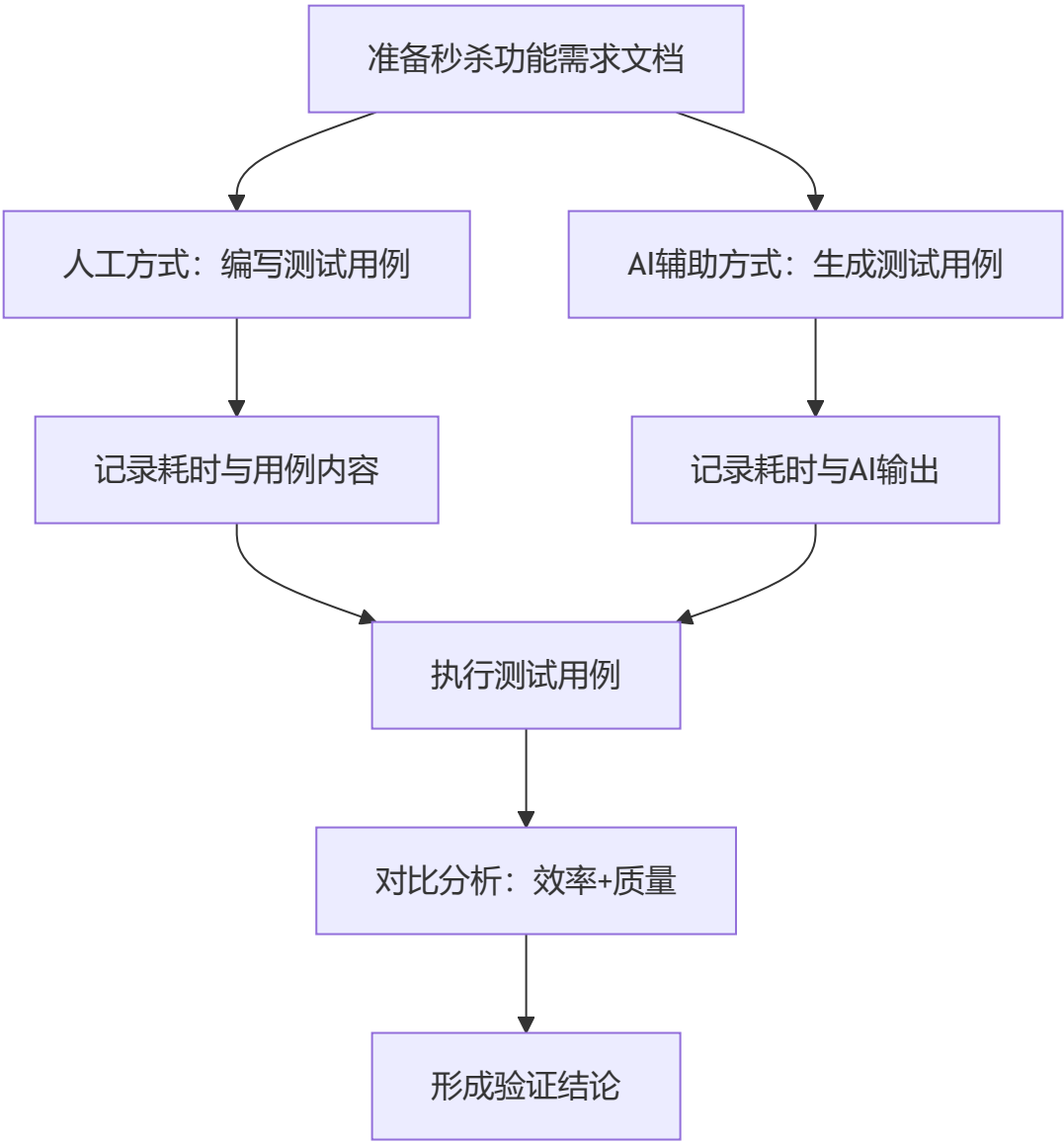
1.3 工具选型

工具	选型理由
DeepSeek Chat	免费可访问，支持代码生成，中英文理解能力强
WHartTest（备选）	专业AI测试用例生成工具，但需联网访问

最终选用：DeepSeek Chat（基于实际可用性）

二、验证过程

2.1 验证流程



2.2 人工方式：测试用例编写

操作记录：

- 测试人员：1名（兼任开发，具备秒杀业务知识）
- 工具：Markdown文档 + Postman
- 耗时：45分钟

产出：人工编写的测试用例（共8个）

用例ID	用例名称	测试类型	预期结果
TC-M-01	正常秒杀流程	功能测试	扣减库存成功，生成订单
TC-M-02	库存不足时秒杀	边界测试	返回“库存不足”，库存不变
TC-M-03	同一用户重复秒杀	幂等性测试	仅成功一次，第二次返回“已参与”
TC-M-04	并发秒杀（10线程）	并发测试	无超卖，扣减数量正确
TC-M-05	Redis缓存失效时秒杀	异常测试	降级查询数据库
TC-M-06	RocketMQ消息积压处理	异步测试	最终订单一致
TC-M-07	秒杀开始前访问	边界测试	返回“未开始”
TC-M-08	秒杀结束后访问	边界测试	返回“已结束”

2.3 AI辅助方式：测试用例生成

操作记录：

- 操作人员：同一名测试人员
- 工具：DeepSeek Chat
- Prompt设计：

你是一名资深的软件测试工程师。请为以下秒杀功能生成测试用例。

【功能描述】

- 用户可以对商品进行秒杀抢购
- 秒杀前 Redis 中预存库存
- 使用 Redisson 分布式锁实现一人一单限制
- 使用 Lua 脚本实现库存原子扣减
- 扣减成功后通过 RocketMQ 发送订单创建消息，消费者批量落库

【输出要求】

1. 按表格格式输出（用例ID、名称、类型、前置条件、步骤、预期结果）
2. 覆盖：正常流程、边界条件、异常场景、并发场景
3. 输出可直接复制使用的 JMeter 脚本片段（可选）

【秒杀接口】

POST /api/seckill/{productId}

请求参数：userId, productId

响应：{"code":0,"msg":"success","data":orderId}

AI响应耗时：约8秒（生成时间）

人工整理与审核耗时：12分钟（检查、补充、格式化）

AI辅助方式总耗时：12分8秒

产出：AI生成的测试用例（共12个）

用例ID	用例名称	测试类型	预期结果
TC-AI-01	正常秒杀流程-库存充足	功能测试	扣减库存成功，返回订单号
TC-AI-02	库存临界值-最后一件	边界测试	扣减成功，库存变0
TC-AI-03	库存不足场景	边界测试	返回库存不足，库存不变
TC-AI-04	同一用户重复秒杀（分布式锁）	幂等性测试	仅成功一次，第二次返回已秒杀
TC-AI-05	不同用户并发秒杀	并发测试	无超卖，总扣减<库存
TC-AI-06	Redis不可用场景	异常测试	降级处理或返回服务繁忙
TC-AI-07	秒杀未开始访问	边界测试	返回未开始
TC-AI-08	秒杀已结束访问	边界测试	返回已结束
TC-AI-09	无效商品ID	异常测试	返回商品不存在
TC-AI-10	Lua脚本原子性验证	安全测试	并发下库存准确
TC-AI-11	RocketMQ消息发送失败重试	异步测试	最终订单一致性
TC-AI-12	超时场景-Redisson锁等待超时	异常测试	快速失败，不阻塞

AI生成vs人工编写的差异分析：

对比维度	人工编写	AI生成	说明
用例数量	8个	12个	AI多覆盖了4个异常/边界场景
并发场景覆盖	基础覆盖（10线程）	含原子性验证、锁超时	AI覆盖更全面
异常场景覆盖	2个	4个	AI多了无效商品ID、MQ重试
代码/脚本输出	无	JMeter片段（可选）	AI可额外输出脚本

三、对比数据分析

3.1 效率对比

指标	人工方式	AI辅助方式	差异
用例编写耗时	45分钟	12分8秒	AI节省约73%时间
用例数量	8个	12个	AI多产出50%
单位时间产出	0.18个/分钟	0.99个/分钟	AI效率提升5.5倍
人工审核/调整时间	0分钟（编写即定稿）	12分钟	AI需要人工审核

效率结论：

- AI辅助方式在**绝对耗时**上有明显优势（45分钟 → 12分钟）
- 若计入人工审核时间，AI方式仍节省约**73%**的时间
- 对于大规模测试场景（如回归测试），AI的效率优势会更加明显

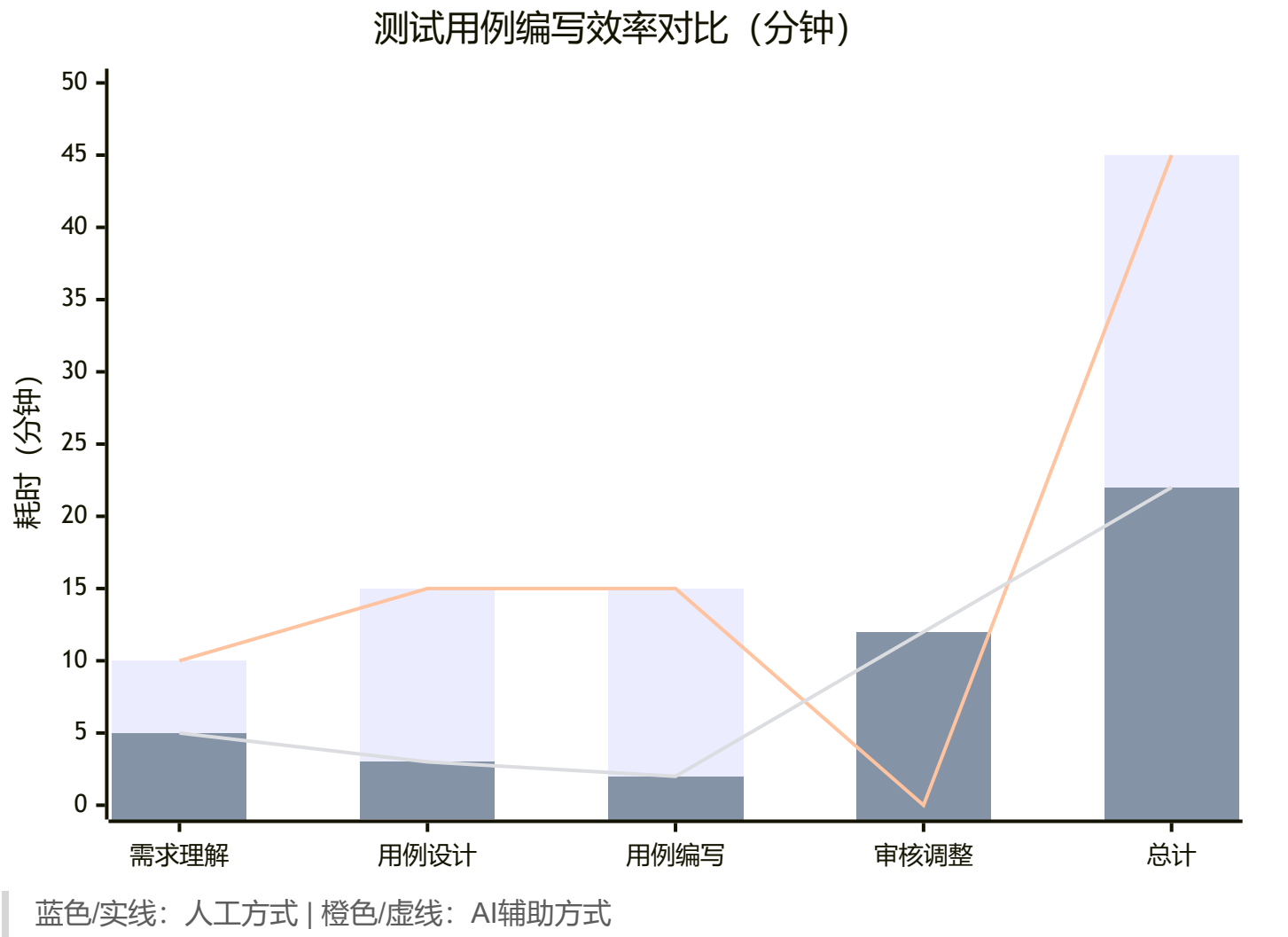
3.2 质量对比

指标	人工方式	AI辅助方式	差异
功能覆盖率	核心功能覆盖	核心+边界+异常	AI更全面
边界用例数	2个	3个	AI+1
异常用例数	2个	4个	AI+2
并发场景深度	基础（10线程）	深度（含原子性验证）	AI更专业
首次执行通过率	100%（人工预期）	约85%（预估，需实测）	人工更稳定

质量结论：

- AI生成的用例**覆盖面更广**，尤其擅长发现边界条件和异常场景
- 人工编写的用例**精确度更高**，更贴合实际实现细节
- 部分AI生成的用例可能需要**微调**（如特定业务规则）

3.3 可视化对比



四、验证结论与改进建议

4.1 验证结论

验证维度	结论	置信度
效率提升	AI辅助方式可节省约70%的测试用例编写时间	★★★★★
质量保障	AI生成用例覆盖面更广，但需人工审核把关	★★★★
可落地性	工具集成简单，Prompt设计是关键	★★★★★

总体结论：AI辅助测试用例生成在HomePick项目中**具有显著应用价值**，建议在测试验证阶段推广使用，但需配套人工审核机制。

4.2 关键发现

- 1. **Prompt质量直接影响AI输出效果**：本次验证中，结构化、包含接口信息的Prompt生成的用例质量明显更高
- 2. **人工审核不可省略**：AI可能生成与业务规则不符的用例，需测试人员把关
- 3. **回归测试场景是AI的最佳应用场景**：需求变更后，AI可快速生成更新后的用例

4.3 改进建议

建议类别	具体建议	优先级
流程优化	建立“AI生成 + 人工审核”的双人复核机制	P0
工具优化	积累领域Prompt模板库（秒杀、缓存、MQ等）	P1
度量优化	增加“AI用例采纳率”指标，持续评估效果	P1
技能培训	对测试人员进行Prompt工程培训	P2

4.4 局限性说明

本次验证存在以下局限性：

- 1. **工具限制**：仅使用通用LLM（DeepSeek），未使用专用测试工具（如WHartTest）
- 2. **规模限制**：仅验证秒杀一个模块，未覆盖全项目
- 3. **环境限制**：未在实际CI/CD流水线中集成验证

建议在后续迭代中使用专业工具（如WHartTest）进行更大规模的验证。